

Building a Full Machine Learning Pipeline for Housing Price Prediction

Data Cleaning → Training → Evaluation → Deployment

Introduction

This guide explains how to build a complete machine learning pipeline for predicting housing prices in British Columbia. It walks through each step required to go from raw data to a working prediction system.

The goal is to provide a clear, reproducible workflow that other students can follow and apply to similar regression problems.

The pipeline includes:

- Data cleaning
- Preprocessing
- Model training
- Hyperparameter tuning
- Evaluation
- Pipeline orchestration
- Prediction on new data

Step 1: Data Cleaning

Before training any model, clean the dataset to remove unreliable or inconsistent data.

Run:

```
python data_cleaning.py
```

This will filter the dataset to British Columbia only, remove 14 sparse columns with many missing values, and removes unrealistic values using thresholds:

Feature	Range
Price	100,000 to 5,000,000
Square Footage	200 to 10,000
Bedrooms	1 to 10
Bathrooms	1 to 10

Example Code:

```
bc_df = df[df["Province"] == "BC"]

drop_cols = [
    "Garage", "Parking", "Basement", "Exterior", "Fireplace",
    "Heating", "Flooring", "Roof", "Waterfront", "Sewer",
    "Pool", "Garden", "Balcony", "Acreage"
]
bc_df = bc_df.drop(columns=drop_cols, errors="ignore")
```

Output:

```
cleaned_bc_data.csv
```

Step 2: Data Preprocessing

Machine learning models require numerical input and consistent feature scaling.

Run:

```
python preprocessing.py
```

This will split data into training and test sets, apply StandardScaler for numerical features, and OneHotEncoder for categorical features.

Ensure you split the data before fitting transformations to avoid data leakage.

Example:

```
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numerical_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Figure 1

Console output after running preprocessing.py. Note the column explosion caused by one-hot encoding

```
Dataset shape: (21977, 9)
Categorical columns: ['City', 'Property Type']
Numerical columns: ['Latitude', 'Longitude', 'Bedrooms', 'Bathrooms', 'Square Footage']
Training data shape: (17581, 243)
Test data shape: (4396, 243)
```

From Figure 1, we can see that the one-hot encoding is working as intended. Each unique City and Property Type gets its own binary column and this is how we give a regression model the ability to learn city-specific price effects.

Step 3: Model Training

We trained six regression models to compare performance across different assumptions:

- Linear, Ridge, and Lasso regression
- Decision Tree
- Random Forest
- K-Nearest Neighbours

Each model captures different relationships in the data. Comparing models helps identify which structure best fits the dataset.

Run:

```
python milestone2_models.py
```

Example:

```
rf_model = Pipeline([
    ("preprocess", preprocessor),
    ("model", RandomForestRegressor()),
])

rf_model.fit(X_train, y_train)
```

This removes the need for manual preprocessing and prevents inconsistencies.

This will save model files such as random_forest.pkl.

Step 4: Hyperparameter Tuning

Default model parameters are not optimized for specific datasets. We used GridSearchCV with cross-validation to find better configurations.

Cross-validation splits the training data into multiple folds and evaluates each configuration across all folds. This provides a more reliable estimate of performance.

Run:

```
python tuning.py
```

This will use GridSearchCV with 3-fold cross-validation, test multiple parameter combinations, and select the best model based on performance.

4.1 What is cross-validation and why use it?

We cannot evaluate hyperparameter combinations on the test set because that would allow the hyperparameters to overfit to the test data, defeating the purpose of holding it out. Instead, we use k-fold cross-validation on the training data by splitting it into three folds, training on two folds, evaluating on the third, rotating through all folds, and averaging the RMSE scores. This provides a fair estimate of how each hyperparameter combination will generalize.

Figure 2

Hyperparameter Tuning Results

```
Best params for Random Forest: {'model__max_depth': 20, 'model__min_samples_leaf': 1, 'model__min_samples_split': 2, 'model__n_estimators': 200}
Best RMSE: 415260.5521

Saved: best_random_forest.pkl
```

Note. This is only the console output for Random Forest for space consideration. There will be other models appearing as well.

Figure 2 shows `max_depth = 20` (not `None`) is significant as unconstrained trees memorize the training data. Capping depth at 20 forces the model to find generalizable splits rather than fitting noise. This is a form of regularization for tree-based models.

Step 5: Evaluating and Interpreting Results

A single accuracy number is rarely enough. We used four complementary views to understand not just which model is best, but why, and where it still fails.

Run:

```
python evaluation.py
```

5.1 The three metrics and what they tell you

- **MAE:** average prediction error
- **RMSE:** penalizes large errors more heavily
- **R²:** proportion of variance explained

Random Forest achieved the best performance with an R² of approximately 0.81.

Figure 3*Model Performance Comparison*

Model Performance Comparison (↑ green = best RMSE)

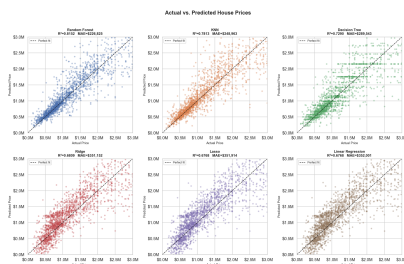
Model	MAE	RMSE	R ²
Random Forest	\$226,825	\$404,287	0.8152
KNN	\$248,963	\$439,905	0.7813
Decision Tree	\$289,543	\$489,652	0.7290
Ridge	\$351,152	\$531,322	0.6809
Lasso	\$351,914	\$534,720	0.6768
Linear Regression	\$352,001	\$534,738	0.6768

Figure 3 shows all six models on the held-out test set. The gap between Random Forest and linear models confirms nonlinear relationships in the data.

5.2 Actual vs Predicted Scatter Plots

For each model, we plot predicted price (y-axis) against actual price (x-axis). A perfect model would produce all points on the dashed diagonal. Deviations from the diagonal reveal systematic biases:

- Predictions cluster below the diagonal at high prices, indicating that the model tends to underestimate expensive homes.
- There is a wider spread at higher price ranges compared to lower ones, showing that the model is less consistent and less confident when predicting expensive properties (heteroscedasticity).
- Random Forest shows the tightest clustering around the diagonal, especially in the \$200K–\$1.5M range where most BC listings fall

Figure 4*Actual vs Predicted Housing Prices*

This figure compares predicted and actual house prices across models. Random Forest performs best, with predictions closest to the ideal diagonal line. KNN performs reasonably well but shows more variation at higher prices, while Decision Trees are more scattered, indicating weaker generalization. Linear models perform worst, with larger errors and inability to capture nonlinear patterns. Across all models, high-value properties are consistently underpredicted, and errors increase as price increases.

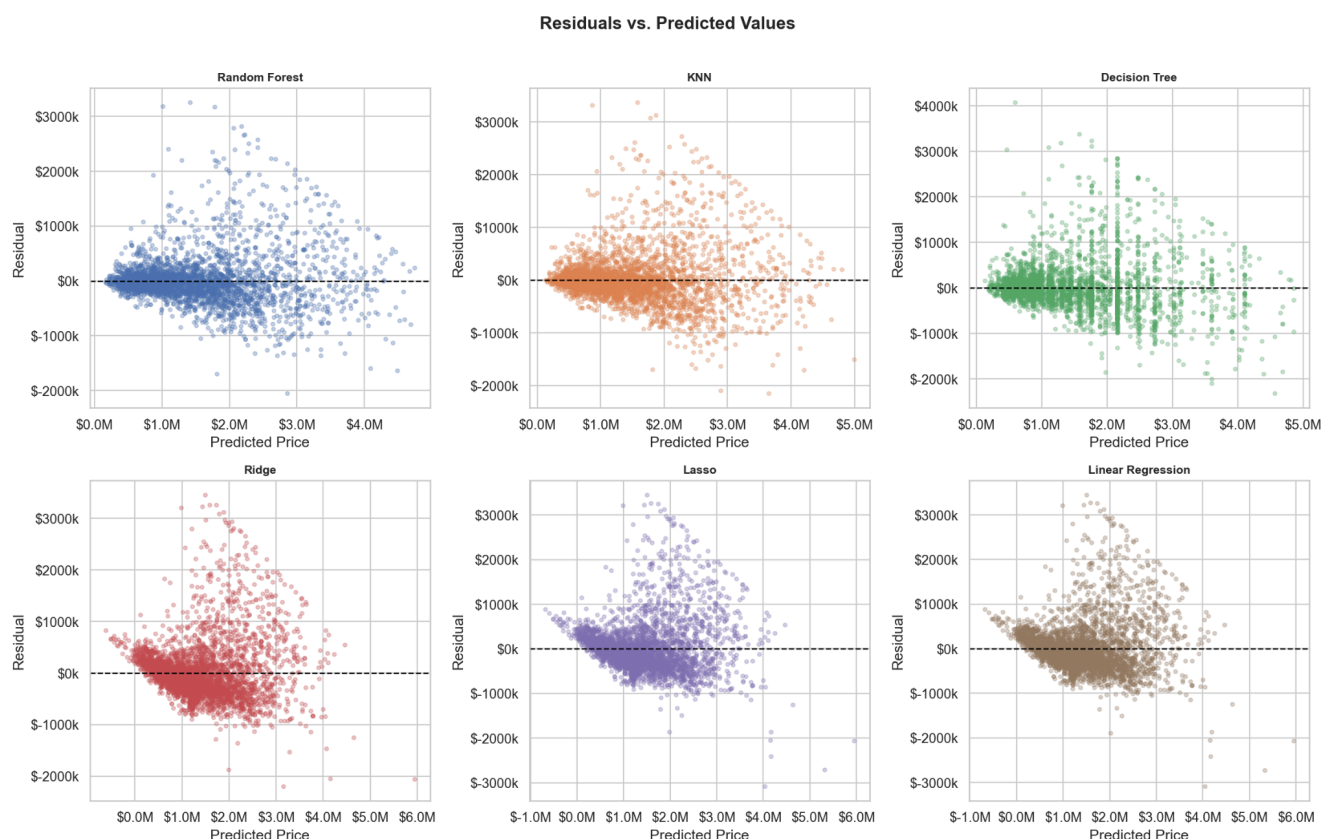
5.3 Residual Distributions

A residual is the difference between actual and predicted price: (actual - predicted). Plotting residuals as a histogram tells you the shape of your errors:

- A narrow, symmetric distribution centred at zero means errors are small and random, indicating strong and reliable model performance.
- A distribution shifted left or right means the model has a systematic bias (consistently over- or under-predicts)
- A long right tail means the model occasionally makes catastrophically large underestimates

Figure 5

Residual Distributions



This figure shows residuals versus predicted prices for each model. Random Forest has the most balanced distribution around zero, indicating fewer systematic errors. KNN and Decision Tree show greater spread, especially at higher prices, while linear models exhibit bias and increasing error. Across all models, error variance increases with price, indicating lower reliability for expensive properties.

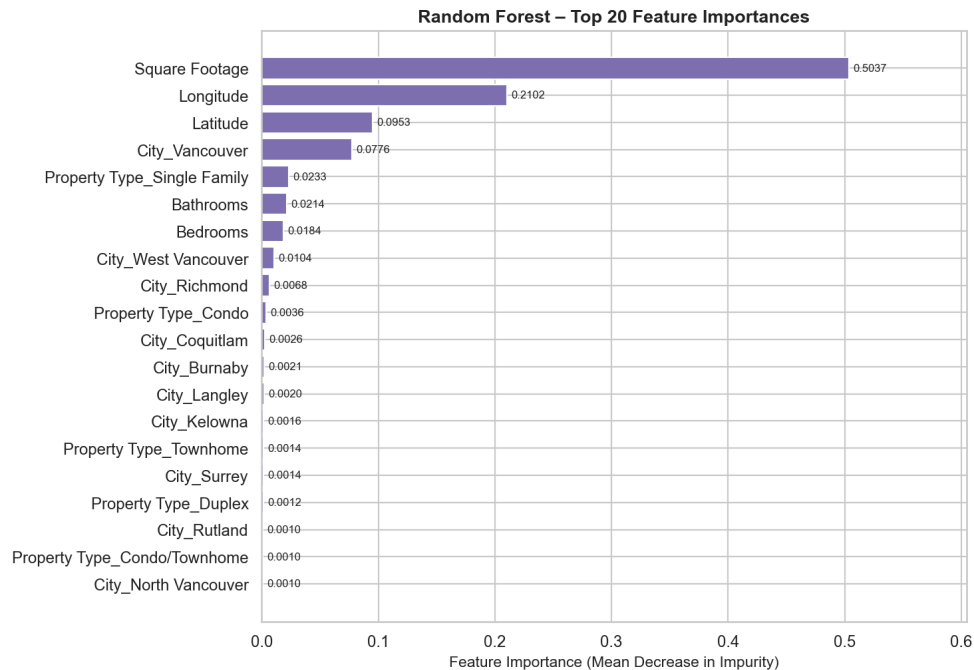
5.4 Feature importance

License: The authors grant Simon Fraser University permission to share this guide for academic use. All copyright remains with the original authors.

Random Forest can report how much each feature contributed to reducing prediction error across all trees. This is called Mean Decrease in Impurity (MDI).

Figure 6

Feature Importance



The dominance of Latitude and Longitude is a useful finding: geographic coordinates capture price gradients (coastal, urban, suburban) more precisely than city names, because the “Vancouver” one-hot column treats all Vancouver listings the same regardless of neighbourhood.

5.5 Error analysis

Beyond average error, we identified the top 1% of largest prediction errors and saved them for inspection.

Figure 4

Error Analysis Output

City	Latitude	Longitude	Bedrooms	Bathroom	Property T	Square Fo	Actual Pri	Predicted	Residual	Absolute Error
Coquitlam	49.25259	-122.89	1	1	Single Fan	600	4670050	1412299	3257751	3257751
Maple Rid	49.20695	-122.551	3	3	Townhom	1906	4190000	1007396	3182604	3182604
Langley	49.06045	-122.657	2	2	Single Fan	1400	4950000	1777212	3172788	3172788
Victoria	48.42222	-123.385	2	4	Condo	4155	5000000	2176344	2823656	2823656
Maple Rid	49.22534	-122.648	3	2	Single Fan	3038	4850000	2058592	2791408	2791408
West Vanc	49.36494	-123.285	2	2	Single Fan	2366	4888000	2222929	2665071	2665071
Surrey	49.16943	-122.804	5	2	Single Fan	2600	4875000	2211944	2663056	2663056
Surrey	49.1925	-122.82	4	2	Single Fan	2380	4990000	2416225	2573775	2573775
Lake Cour	50.01627	-119.441	4	5	Single Fan	4332	4789000	2283464	2505536	2505536
Coquitlam	49.27238	-122.805	2	1	Single Fan	1088	4850000	2418172	2431828	2431828

Note. Full CSV file can be found attached.

Inspecting these rows reveals failure patterns such as luxury homes, rare property types, or underrepresented locations. The largest errors often occur where training data is limited or feature combinations are uncommon. This analysis helps identify where additional data or feature improvements are needed.

Step 6: Orchestrating the Full Pipeline

Once each script works independently, the next step is coordinating execution without repeating expensive steps. We implemented an orchestration script that checks for existing outputs before running each stage.

Run:

```
python run_pipeline.py
```

This will check if model files already exist, skips training if models are already saved, and runs evaluation and prediction automatically.

Figure 5

Pipeline Execution Output

```
Starting full pipeline...
Model pickle files found. Skipping training step.
=====
Running: evaluation.py
=====
Random Forest      MAE=$226,825  RMSE=$404,287  R2=0.8152
KNN                MAE=$248,963  RMSE=$439,905  R2=0.7813
Decision Tree      MAE=$289,543  RMSE=$489,652  R2=0.7290
...
Best model: Random Forest (RMSE=$404,287, R2=0.8152)
Pipeline completed successfully.
```

Note. This is a heavily trimmed version of the console output for space considerations.

Step 7: Making a Prediction on a New Property

The final test of a pipeline is whether it can handle new, unseen input and produce a reasonable prediction. Our demo.py does this by loading the saved Random Forest pipeline and calling predict() on raw feature data.

This works without manual preprocessing because the .pkl file contains both the preprocessor and the model. The pipeline automatically handles encoding and scaling, allowing raw inputs to be used directly.

Run:

```
python demo.py
```


This loads the trained Random Forest pipeline, accepts raw input features, and outputs the predicted price.

Troubleshooting

Issue 1: Missing Model File

Error: model file not found

Solution: run tuning.py to generate required files

Issue 2: Lasso Convergence Warning

Cause: high dimensional feature space

Solution: increase max_iter

Issue 3: Poor KNN Performance

Cause: high dimensionality reduces distance reliability

Solution:

- ensure proper scaling
- consider dimensionality reduction
- prefer ensemble models

Conclusion

This pipeline demonstrates the importance of structured machine learning workflows. Data must be cleaned before modeling, and preprocessing should be performed after splitting to prevent data leakage. Pipelines ensure consistency and simplify deployment.

Training multiple models revealed significant performance differences, with Random Forest outperforming linear approaches. Evaluation beyond summary metrics provided deeper insight into model limitations, particularly for high-value and underrepresented properties.

All scripts are included in the submission to support reproducibility and further exploration.